RAG

[Document] → Parsing → cleaning → chunking → Embedding → Index → Retrieval → Reranking → prompt Construction → Generations → citation validation

RAG Pipeline consists of 12 distinct, sequential Phases. Engineering Faliures can occure at any of thes steps

Phase	Description
Parsing	Extracting raw text from diverse source formats (PDFs, Markdown, Word, HTML, JSON).
Cleaning	Stripping boilerplate code, headers, footers, and formatting noise.
Chunking	Splitting raw continuous text into discrete segments.
Embedding	Transforming text chunks into high-dimensional dense vectors via an embedding model.
Indexing	Storing vectors and token indices in a database for rapid nearest-neighbor or inverted-index lookups.
Retrieval	Querying the database to fetch matching document chunks based on user input.
Reranking	Re-scoring the first-pass retrieved pool with a more computationally heavy cross-encoder.
Prompt Construction	Injecting the final, selected chunks cleanly into the LLM context window alongside the user query.
Generation	Processing the engineered prompt to output a response.
Citation Validation	Programmatically cross-referencing output text against the actual source context to verify citations.
Evaluation	Calculating system-level metrics (e.g., context recall, faithfulness) to ensure quality.

Technical Mechanics

Chunking strategy

Controlles exatcy what the retriever can find

Bad chunking risks

- missing context
- fracturing concepts
- Noise

Strategys

- Fixed-size: Slicing text by token count
  - No Semantic
- Semantic: spliting based on shift points in semantic vector samilirty
- Section-based: Respecting layouts (headers, tabels, ...etc)
- Parent-child (Hierarchical): Slicing text into small child chunks for precise vector matching, but retriving bordar parent for richer context

RRF & Hybrid Retrieval

Relying on vector embeddings alone fails on precise techinal

→ Production architectures use Hybrid Retrieval to combine lexical Semantic matching:

* BM25 (lexical) : Evalutes exact key word matches, phrases frequency, rare tokens

* Dense (semantic) : Evalutes abstract semantic intent and conceptual catching, relevent text even if zero keywords align

to combine these separate ranking lists cleanly, use Reciprocal Rank Fusion (RRF)

$$RRF\_\delta core(d \in D) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

(where M is the set of retrieval systems, $r_m(d)$ is the rank of document d in system m, and K is a constant smoothing factor typicall set around 60)

Reranking

⟹ Vector searches for high recall, grab every thing Vaguely relevent, Reranker optimizes for high Precision

* Flow: Retiever a Large initial Pool (e.g, top 50 via (Hybrid RRF), run them in cross-encoder Reranker to compute Fin-graind query-document similarity, and Filter to the top 10-15 chunks

* Trade-off: ↑ Final generation accuracy, NEW Latency Penalty

Systematic Debugging

Failure Mode	Root Cause Analysis	Corrective Action
Missing Context	Chunks exist in the corpus, but the retriever missed them completely. MD	Re-evaluate embedding models, lower the BM25/Dense ratio, or increase first-pass retrieval limits. MD
Fragmented Answers	True context was spread across boundaries, causing the LLM to get an incomplete picture. MD	Transition to a parent-child chunking model or increase the chunk overlap window. MD
Irrelevant Retrieval	The retriever fetched unrelated text, filling the LLM context with noise. MD	Implement or tune a Cross-Encoder reranker to strip out low-relevance documents. MD
Hallucination	System retrieved the exact right context, but the LLM still made up facts during generation. MD	Tighten system instructions, enforce strict grounding rules, or lower model generation temperature. MD
Citation Mismatch	Output claims a fact but links it to an completely unrelated source chunk. MD	Implement an automated regex or NLI post-processor to map output sentences to source chunk IDs. MD